



LABORATORIO N° 12

DOCENTE: Pilar Rocío Sayán Mejía
CURSO: Minería de Datos

Semana12:

Métodos Probabilísticos: Naïve Bayes
Clasificación de SMS Spam



Pilar Rocío Sayán Mejía

Minería de Datos

Tema: Métodos Probabilísticos: Naïve Bayes

Laboratorio: Clasificación de SMS Spam

I. Elemento de la capacidad terminal de la semana

Comprende los fundamentos de los modelos probabilísticos de clasificación. Aplica el algoritmo Naïve Bayes (Multinomial y Bernoulli) a la clasificación de texto en lenguaje natural. Implementa un pipeline completo de procesamiento NLP (limpieza, tokenización, eliminación de stopwords, TF-IDF) para detectar mensajes spam. Compara modelos con métricas de clasificación (Accuracy, Precision, Recall, F1-Score, AUC-ROC) y evalúa su aplicabilidad en sistemas de filtrado de correo no deseado.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red sin supervisión.
- Prohibida la ingesta de alimentos y bebidas dentro del laboratorio.
- Ubicar maletines y/o mochilas en el lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas al terminar.
- No descargar software no autorizado en los equipos del laboratorio.
- Todo código se ejecuta únicamente en Google Colab; no instalar paquetes en la máquina local.

III. Fundamento Teórico

Antes de realizar este laboratorio, el estudiante debe haber revisado el siguiente material:

- Diapositivas de la Semana 12: Métodos Probabilísticos y Naïve Bayes.
- James, G., Witten, D., Hastie, T. y Tibshirani, R. (2023). An Introduction to Statistical Learning (2.^a ed.). Springer. Capítulo 4.

- Géron, A. (2022). Hands-On Machine Learning (3.^a ed.). O'Reilly Media. Capítulo 3.
- Müller, A. C. y Guido, S. (2016). Introduction to Machine Learning with Python. O'Reilly Media. Capítulo 6 — Text Data.

Conceptos clave: Teorema de Bayes, probabilidad condicional, independencia condicional, Naïve Bayes Multinomial, Naïve Bayes Bernoulli, Bag of Words, tokenización, stopwords, TF-IDF, clasificación de texto, NLP, spam detection.

IV. Recursos

- Computadora con acceso a internet y navegador web.
- Cuenta de Google para acceder a Google Colab.
- Dataset: SMS Spam Collection Dataset — UCI Machine Learning Repository / Kaggle (5,574 mensajes etiquetados como ham/spam).
- Librerías de Python: pandas, numpy, matplotlib, seaborn, scikit-learn (preinstaladas en Colab).

V. Metodología para el desarrollo de la tarea

El desarrollo del laboratorio es individual.

Duración estimada: 2 horas (1 sesión de laboratorio).

Entregable: Notebook ejecutado de Google Colab (.ipynb) con código ejecutado y respuestas completas a todas las preguntas.

VI. Procedimiento

ACTIVIDAD 1: Revisión de Conceptos — Modelos Probabilísticos y NLP

Instrucción: Complete la siguiente tabla con definiciones propias basadas en lo estudiado en la clase teórica. No copie textualmente de los materiales; use sus propias palabras.

N°	Concepto / Principio	Definición con sus propias palabras
1	Naïve Bayes	
2	Teorema de Bayes	
3	Probabilidad condicional	
4	Independencia condicional	
5	Bag of Words	
6	Tokenización	
7	Stopwords	
8	TF-IDF	
9	Multinomial Naïve Bayes	
10	Bernoulli Naïve Bayes	
11	Clasificación de texto	

N°	Concepto / Principio	Definición con sus propias palabras
12	¿Por qué Naïve Bayes funciona bien en NLP?	

ACTIVIDAD 2: Desarrollo Práctico — Clasificación de SMS Spam

Ref: Géron, A. (2022). Hands-On Machine Learning, Cap. 3 / James et al. (2023). ISLR, Cap. 4.

En esta actividad construirás un pipeline completo de clasificación de texto: desde la carga del dataset SMS Spam hasta la comparación de tres modelos de clasificación. Sigue cada paso en orden; el código ya está listo en el notebook de Google Colab adjunto.

PASO 1 — Importación de Librerías y Configuración del Entorno

Configuramos el entorno importando todas las librerías necesarias. Definimos estilos de visualización coherentes para el laboratorio.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import string

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report, roc_curve, auc)
```

PASO 2 — Carga del Dataset SMS Spam

Usamos el SMS Spam Collection Dataset con 5,574 mensajes de texto etiquetados como ham (mensaje normal) o spam (mensaje no deseado). El dataset proviene del UCI Machine Learning Repository.

Variable objetivo: label → ¿Es el mensaje spam? (ham/spam)

```
url = 'https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv'
df = pd.read_table(url, header=None, names=['label', 'message'])

print(f'Dataset cargado: {df.shape[0]:,} filas x {df.shape[1]} columnas')
print(df['label'].value_counts())
print(f'\nDistribucion:')
print(df['label'].value_counts(normalize=True).round(4) * 100)
```

 **Pregunta 1:** ¿Cuántos mensajes spam existen en el dataset? ¿El dataset está balanceado? ¿Qué porcentaje representa el spam? ¿Por qué el desbalance de clases puede ser un problema?

Respuesta:

PASO 3 — Preprocesamiento de Texto

Aplicamos técnicas básicas de limpieza de texto: conversión a minúsculas y eliminación de puntuación. Un texto limpio reduce el vocabulario y mejora la generalización del modelo.

```
def clean_text(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text

df['message_clean'] = df['message'].apply(clean_text)
df[['message', 'message_clean']].head(5)
```

 **Pregunta 2:** ¿Por qué es importante limpiar el texto antes de entrenar un modelo? ¿Qué ocurriría si no eliminamos puntuación ni convertimos a minúsculas?

Respuesta:

PASO 4 — Tokenización y Bag of Words (BoW)

Aplicamos CountVectorizer para convertir los mensajes de texto en una matriz numérica donde cada columna representa una palabra del vocabulario y cada celda indica la frecuencia de esa palabra en el mensaje.

```
X_text = df['message_clean']
y = df['label']

# 70% train+validacion y 30% test final.
X_trainval_text, X_test_text, y_trainval, y_test = train_test_split(
    X_text, y, test_size=0.30, random_state=42, stratify=y
)

# Del 70% restante separamos validacion para elegir modelo sin mirar test.
X_train_text, X_val_text, y_train, y_val = train_test_split(
    X_trainval_text, y_trainval, test_size=0.20, random_state=42, stratify=y_trainval
)

vectorizer_bow = CountVectorizer(stop_words='english', max_features=10000)
X_bow_train = vectorizer_bow.fit_transform(X_train_text)
X_bow_val = vectorizer_bow.transform(X_val_text)
X_bow_test = vectorizer_bow.transform(X_test_text)

print(f'Vocabulario: {len(vectorizer_bow.vocabulary_):,} palabras unicas')
print(f'Train BoW: {X_bow_train.shape[0]:,} mensajes x {X_bow_train.shape[1]:,} palabras')
print(f'Validacion BoW: {X_bow_val.shape[0]:,} mensajes')
print(f'Test BoW: {X_bow_test.shape[0]:,} mensajes')
```

 **Pregunta 3:** ¿Qué representa cada columna en la matriz Bag of Words? ¿Por qué la matriz es altamente dispersa (sparse)? ¿Qué información pierde la representación BoW?

Respuesta:

PASO 5 — Representación TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) pondera cada término según su frecuencia en el documento y su rareza en el corpus. Palabras muy comunes reciben menor peso; palabras raras y discriminativas reciben mayor peso.

```
tfidf = TfidfVectorizer(stop_words='english', max_features=5000, ngram_range=(1, 2))
X_train = tfidf.fit_transform(X_train_text)
X_val = tfidf.transform(X_val_text)
X_test = tfidf.transform(X_test_text)

print(f'Train TF-IDF: {X_train.shape[0]:,} mensajes x {X_train.shape[1]:,} terminos')
print(f'Validacion TF-IDF: {X_val.shape[0]:,} mensajes x {X_val.shape[1]:,} terminos')
print(f'Test TF-IDF: {X_test.shape[0]:,} mensajes x {X_test.shape[1]:,} terminos')

idf_scores = tfidf.idf_
top_idx = np.argsort(idf_scores)[-10:][::-1]
print('Top 10 terminos con mayor IDF:', [tfidf.get_feature_names_out()[i] for i in
top_idx])
```

 **Pregunta 4:** ¿Cuál es la diferencia entre Bag of Words y TF-IDF? ¿Por qué TF-IDF es generalmente mejor para clasificación de texto? Da un ejemplo concreto.

Respuesta:

PASO 6 — División Train / Test

Dividimos los datos en 70% entrenamiento y 30% prueba. El parámetro stratify garantiza que la proporción de spam/ham se preserve en ambos conjuntos, evitando particiones sesgadas.

División Holdout 70/30 con estratificación

70% Train → el modelo aprende los patrones de spam/ham (~3,902 mensajes aprox.)

30% Test → evaluamos generalización a mensajes no vistos (~1,672 mensajes aprox.)

stratify=df['label'] → preserva la proporción ham/spam en train y test


```
print(f'Train: {X_train.shape[0]:,} | Validacion: {X_val.shape[0]:,} | Test: {X_test.shape[0]:,}')
print('Proporcion de clases en train:')
print(y_train.value_counts(normalize=True).round(4) * 100)
print('Proporcion de clases en validacion:')
print(y_val.value_counts(normalize=True).round(4) * 100)
print('Proporcion de clases en test final:')
print(y_test.value_counts(normalize=True).round(4) * 100)
```

PASO 7 — Entrenamiento de Modelos

Entrenamos tres modelos con los datos de entrenamiento. Ref: Géron, A. (2022). Hands-On Machine Learning. Cap. 3.

Multinomial Naïve Bayes	Bernoulli Naïve Bayes	Regresión Logística
Trabaja con frecuencias de palabras. Ideal para texto con TF-IDF o BoW.	Trabaja con presencia/ausencia de palabras. Útil para documentos cortos.	Baseline discriminativo. Línea base sólida para clasificación de texto.

```
models = {
    'Multinomial NB': MultinomialNB(alpha=0.1),
    'Bernoulli NB': BernoulliNB(alpha=0.1),
    'Logistic Reg.': LogisticRegression(max_iter=1000, C=1.0)
}

trained_models = {}
for nombre, modelo in models.items():
    modelo.fit(X_train, y_train)
    trained_models[nombre] = modelo
    print(f'{nombre}: entrenado')
```

PASO 8 — Evaluación de Modelos

Calculamos las métricas de clasificación para los tres modelos. Ref: James et al. (2023). ISLR. Cap. 4.

Métrica	Fórmula	Interpretación / Cuándo usar
Accuracy	$(VP+VN)/(VP+VN+FP+FN)$	Clases balanceadas
Precision	$VP/(VP+FP)$	Cuando el costo de FP es alto
Recall	$VP/(VP+FN)$	Cuando el costo de FN es alto (spam)
F1-Score	$2 \cdot \text{Prec} \cdot \text{Recall} / (\text{Prec} + \text{Recall})$	Desbalance + balance entre errores
AUC-ROC	Área bajo la curva ROC	Comparar modelos y umbrales

```
def evaluar_modelos(modelos, X_eval, y_eval):
    resultados = {}
    for nombre, modelo in modelos.items():
        y_pred = modelo.predict(X_eval)
        spam_col = list(modelo.classes_).index('spam')
        y_proba = modelo.predict_proba(X_eval)[: , spam_col]
        resultados[nombre] = {
            'Accuracy': accuracy_score(y_eval, y_pred),
            'Precision': precision_score(y_eval, y_pred, pos_label='spam'),
            'Recall': recall_score(y_eval, y_pred, pos_label='spam'),
            'F1-Score': f1_score(y_eval, y_pred, pos_label='spam'),
            'AUC-ROC': roc_auc_score(y_eval == 'spam', y_proba),
        }
    return pd.DataFrame(resultados).T.round(4)

# Se usa validacion para elegir el mejor modelo, no test.
results_df = evaluar_modelos(trained_models, X_val, y_val)
print(results_df)
```

 **Pregunta 5:** ¿Qué modelo obtiene el mejor F1-Score? ¿Y el mejor AUC-ROC? Para un filtro de spam, ¿qué métrica priorizarías: Precision o Recall? Justifica tu respuesta.

Respuesta:

PASO 9 — Matriz de Confusión del Mejor Modelo

Generamos la matriz de confusión para el modelo con mejor F1-Score. Ref: Kelleher, J. D. et al. (2020). Fundamentals of Machine Learning. Cap. 3.

💡 Contexto de negocio — Empresa de Telecomunicaciones

Falso Negativo (FN): Mensaje spam que llega a la bandeja principal → molesta al usuario ✉

Falso Positivo (FP): Mensaje legítimo marcado como spam → pérdida de información importante 🔔

Para spam: FP es extremadamente costoso → maximizar Precision sin sacrificar Recall

```
# Seleccionar el mejor modelo por F1-Score en validacion
best_model_name = results_df['F1-Score'].idxmax()
best_model = trained_models[best_model_name]

# Evaluacion final en test una sola vez
y_pred_best = best_model.predict(X_test)
spam_col_best = list(best_model.classes_).index('spam')
y_proba_best = best_model.predict_proba(X_test)[: , spam_col_best]

cm = confusion_matrix(y_test, y_pred_best, labels=['ham', 'spam'])
print(f'Mejor modelo segun validacion: {best_model_name}')
print(classification_report(y_test, y_pred_best, target_names=['ham', 'spam'],
digits=4))

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['ham', 'spam'], yticklabels=['ham', 'spam'])
plt.title(f'Matriz de Confusion Final - {best_model_name}')
plt.xlabel('Prediccion')
plt.ylabel('Valor Real')
plt.tight_layout()
plt.show()
```

✎ **Pregunta 6:** ¿Qué error es más costoso en detección de spam: un Falso Positivo o un Falso Negativo? ¿Por qué? ¿Cómo ajustarías el umbral de decisión para reducir el error más costoso?

Respuesta:

PASO 10 — Análisis de Palabras más Importantes para Spam

Extraemos las palabras con mayor probabilidad logarítmica de pertenecer a la clase spam según el modelo Multinomial Naïve Bayes. Esto nos permite interpretar el modelo y validar su lógica.

```
mnb = trained_models['Multinomial NB']
feature_names = tfidf.get_feature_names_out()

spam_idx = list(mnb.classes_).index('spam')
ham_idx = list(mnb.classes_).index('ham')

# Diferencia de log-probabilidades: positivo = mas asociado a spam.
spam_score = mnb.feature_log_prob_[spam_idx] - mnb.feature_log_prob_[ham_idx]
top_spam_idx = np.argsort(spam_score)[-20:][::-1]
top_spam_words = [(feature_names[i], spam_score[i]) for i in top_spam_idx]

words, scores = zip(*top_spam_words)
plt.figure(figsize=(10, 5))
plt.barh(words, scores, color='crimson')
plt.xlabel('Log-prob(spam) - Log-prob(ham)')
plt.title('Top 20 terminos mas discriminativos de SPAM')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

 **Pregunta 7:** ¿Qué palabras aparecen con mayor frecuencia en mensajes spam? ¿Son coherentes con lo que esperarías encontrar en un mensaje de spam real? ¿Qué nos dice esto sobre la interpretabilidad del modelo?

Respuesta:

ACTIVIDAD 3: Caso de Estudio — Sistema de Filtrado de Spam Empresarial

Contexto Empresarial

Una empresa de telecomunicaciones recibe millones de SMS diarios en su plataforma de mensajería.

El equipo de Data Science ha construido y evaluado tres modelos de clasificación de spam.

Como analista senior, debes responder cada pregunta citando los resultados numéricos obtenidos en la Actividad 2.

 **Pregunta A:** ¿Por qué Naïve Bayes es adecuado para la clasificación de texto? ¿Qué suposición fundamental hace el modelo y por qué funciona bien en la práctica a pesar de ser aparentemente incorrecta?

Respuesta:

 **Pregunta B:** ¿Qué ventajas tiene Naïve Bayes frente a modelos más complejos como Random Forest o redes neuronales para este problema? Menciona al menos 3 ventajas y 2 desventajas.

Respuesta:

 **Pregunta C:** ¿Cómo mejorarías el modelo usando técnicas modernas de NLP? Describe al menos 3 técnicas (por ejemplo: word embeddings, transformers, BERT) y explica qué mejora aportaría cada una.

Respuesta:

 **Pregunta D:** ¿Qué limitaciones fundamentales tiene Naïve Bayes? ¿En qué tipo de mensajes de spam crees que fallaría más? Da un ejemplo concreto.

Respuesta:

 **Pregunta E:** ¿Cómo implementarías este modelo en producción? Describe el pipeline completo desde la recepción del SMS hasta la decisión de bloqueo, incluyendo consideraciones de escalabilidad, monitoreo y reentrenamiento.

Respuesta:

Conclusiones

Escribe al menos 3 conclusiones técnicas y de aprendizaje. Cada conclusión debe conectar un concepto teórico con la experiencia práctica obtenida en el laboratorio:

1.

2.

3.

Material Complementario y Referencias

1. James, G., Witten, D., Hastie, T. y Tibshirani, R. (2023). An Introduction to Statistical Learning (2.^a ed.). Springer. Capítulo 4.
2. Géron, A. (2022). Hands-On Machine Learning (3.^a ed.). O'Reilly Media. Capítulo 3.

3. Müller, A. C. y Guido, S. (2016). Introduction to Machine Learning with Python. O'Reilly Media. Capítulo 6.
4. Almeida, T. A. y Hidalgo, J. M. G. (2011). SMS Spam Collection Dataset. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/ml/datasets/sms+spam+collection>
5. Kelleher, J. D., Mac Namee, B. y D'Arcy, A. (2020). Fundamentals of Machine Learning (2.^a ed.). MIT Press. Capítulo 3.
6. scikit-learn developers. (2024). scikit-learn User Guide: Naive Bayes. https://scikit-learn.org/stable/modules/naive_bayes.html

Criterios de evaluación



Minería de Datos

Rúbrica holística

Elemento	Comprende modelos probabilísticos y aplica Naïve Bayes para clasificación de texto con NLP.				
Curso	Minería de Datos		Periodo	2026-I	
Actividad	Lab 12 — Clasificación de SMS Spam con Naïve Bayes		Semestre	2026-I	
Alumno			Semana	12	
Docente	Pilar Rocío Sayán Mejía	Fecha		Sección	

Criterios a evaluar	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
1. Revisión de conceptos NLP y probabilísticos (Act. 1)	Todas las definiciones correctas con comprensión profunda de teorema de Bayes y vectorización de texto.	Mayoría correctas con pequeños errores.	Definiciones incompletas o parcialmente correctas.	No completa o definiciones incorrectas.	
2. Preprocesamiento y vectorización TF-IDF (Act. 2 — Pasos 3–5)	Implementa pipeline NLP completo: limpieza, tokenización, BoW y TF-IDF. Interpreta la matriz dispersa.	Implementación correcta con interpretaciones básicas.	Implementación parcial o interpretaciones erróneas.	No implementa o no interpreta resultados.	
3. Entrenamiento y métricas de clasificación (Act. 2 — Pasos 7–8)	Calcula e interpreta Accuracy, Precision, Recall, F1-Score y AUC-	Cálculos correctos con interpretaciones básicas.	Cálculos parciales o interpretaciones erróneas.	No calcula o no interpreta métricas.	

Criterios a evaluar	Excelente	Bueno	Requiere mejora	No aceptable	Puntaje logrado
4. Matriz de confusión y análisis de palabras spam (Act. 2 — Pasos 9–10)	ROC para los tres modelos. Genera matriz de confusión, interpreta FP/FN en contexto y extrae palabras más predictoras de spam.	Genera matriz y análisis básico de palabras.	Matriz incompleta o análisis superficial.	No genera visualizaciones ni comparaciones.	
5. Caso de estudio — Sistema de filtrado de spam (Act. 3)	Evaluación completa con argumentación sólida sobre ventajas de NB, limitaciones y propuesta de mejora con NLP moderno.	Evaluación básica con conexiones generales.	Evaluación incompleta o sin justificación.	No realiza el caso o está vacío.	
Total					

Criterios de evaluación

Acciones a cumplir		Puntaje
Puntualidad y dedicación		1
Cumplimiento de tiempos		1
Conclusiones: ortografía y redacción		1
Puntaje total		

Comentarios respecto del desempeño del alumno

Nivel	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo la mayoría de los requerimientos.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad cumpliendo pocos requerimientos.
No aceptable	No demuestra entendimiento del problema o actividad.